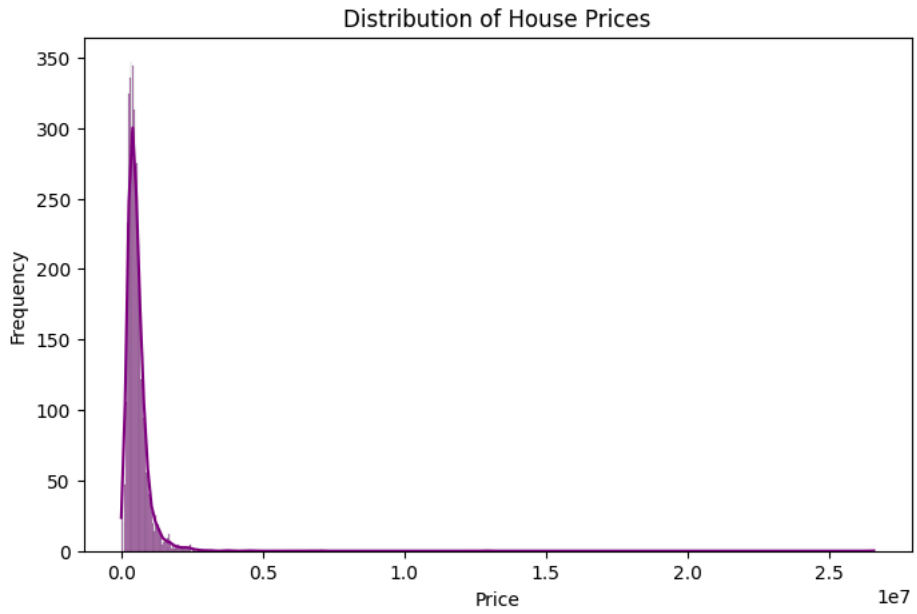
```
import numpy as np    #Pragya Singhal ,CU24220050, 43
import pandas as pd
print('--- Descriptive Statistics ---')
display(df.describe())
```

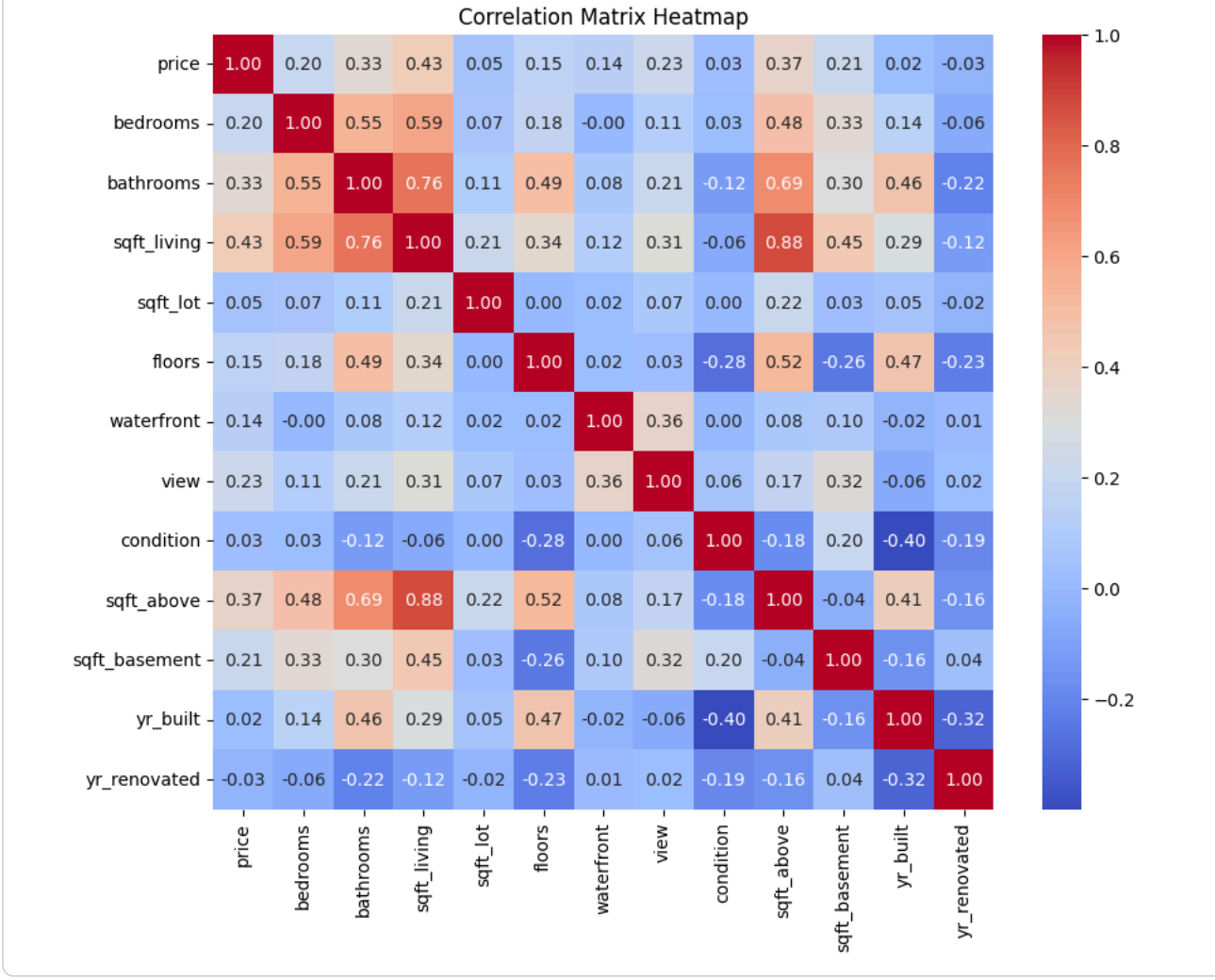
```
--- Descriptive Statistics ---
```

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	
count	4.600000e+03	4600.000000	4600.000000	4600.000000	4.600000e+03	4600.000000	4600.000000	4600.000000	4600.000000	4
mean	5.519630e+05	3.400870	2.160815	2139.346957	1.485252e+04	1.512065	0.007174	0.240652	3.451739	1
std	5.638347e+05	0.908848	0.783781	963.206916	3.588444e+04	0.538288	0.084404	0.778405	0.677230	
min	0.000000e+00	0.000000	0.000000	370.000000	6.380000e+02	1.000000	0.000000	0.000000	1.000000	
25%	3.228750e+05	3.000000	1.750000	1460.000000	5.000750e+03	1.000000	0.000000	0.000000	3.000000	1
50%	4.609435e+05	3.000000	2.250000	1980.000000	7.683000e+03	1.500000	0.000000	0.000000	3.000000	1
75%	6.549625e+05	4.000000	2.500000	2620.000000	1.100125e+04	2.000000	0.000000	0.000000	4.000000	2
max	2.659000e+07	9.000000	8.000000	13540.000000	1.074218e+06	3.500000	1.000000	4.000000	5.000000	9

```
import matplotlib.pyplot as plt     #Pragya Singhal ,CU24220050, 43
import seaborn as sns
plt.figure(figsize=(8, 5))
sns.histplot(df['price'], kde=True, color='purple')
plt.title('Distribution of House Prices')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```




```
import matplotlib.pyplot as plt    #Pragya Singhal ,CU24220050, 43
import seaborn as sns
plt.figure(figsize=(10, 8))
sns.heatmap(
    df.select_dtypes(include=[np.number]).corr(),
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
)
plt.title('Correlation Matrix Heatmap')
plt.show()
```




```
from sklearn.model_selection import train_test_split
columns_to_drop = [     #Pragya Singhal ,CU24220050, 43
    'price',
    'date',
    'street',
    'city',
    'statezip',
    'country',
]
X = df.drop(columns=columns_to_drop)
y = df['price']  # Target variable

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print('Training data shape:', X_train.shape)
print('Testing data shape:', X_test.shape)
```

```
Training data shape: (3680, 12)
Testing data shape: (920, 12)
```

```
from sklearn.ensemble import RandomForestRegressor    #Pragya Singhal ,CU24220050, 43
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

print('Model training completed successfully!')
```

```
Model training completed successfully!
```

```
import numpy as np    #Pragya Singhal ,CU24220050, 43
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
y_pred = model.predict(X_test)
# Calculate performance metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print('--- Model Performance Metrics ---')
print(f'R² Score: {r2 * 100:.2f}%')
```

```
print(f'Mean Absolute Error (MAE): {mae:.2f}')
print(f'Root Mean Squared Error (RMSE): {rmse:.2f}')
```

```
--- Model Performance Metrics ---
R² Score: 2.75%
Mean Absolute Error (MAE): 208109.71
Root Mean Squared Error (RMSE): 995879.44
```